

```

-- =====
-- SCENARIO 1: Customer Shopping Behavior
-- "Which customers are browsing, saving, and buying what"
-- =====

-- 1.1 Show each customer's wishlist with product name and price
SELECT cu.CustomerName, wl.WishListName, p.ProductName,
       pv.Color, pv.Size, vp.SellingPrice
FROM WishList      wl
JOIN WishListItem  wi ON wl.WishListID = wi.WishListID
JOIN Customer      cu ON wl.CustomerID = cu.CustomerID
JOIN ProductVariant pv ON wi.VariantID = pv.VariantID
JOIN Product       p  ON pv.ProductID = p.ProductID
JOIN VendorProduct vp ON pv.VariantID = vp.VariantID
ORDER BY cu.CustomerName;

-- 1.2 Show each customer's cart with total cart value
SELECT cu.CustomerName, p.ProductName, pv.Color, pv.Size,
       ci.Quantity, vp.SellingPrice,
       (ci.Quantity * vp.SellingPrice) AS CartItemTotal
FROM Cart          ca
JOIN CartItem      ci ON ca.CartID      = ci.CartID
JOIN Customer      cu ON ca.CustomerID = cu.CustomerID
JOIN VendorProduct vp ON ci.VP_ID      = vp.VP_ID
JOIN ProductVariant pv ON vp.VariantID = pv.VariantID
JOIN Product       p  ON pv.ProductID = p.ProductID
ORDER BY cu.CustomerName;

-- 1.3 Customers who added to wishlist but never placed any order
SELECT cu.CustomerName, cu.Email, cu.Phone,
       COUNT(wi.WLItemID) AS WishlistItems
FROM Customer      cu
JOIN WishList      wl ON cu.CustomerID = wl.CustomerID
JOIN WishListItem  wi ON wl.WishListID = wi.WishListID
WHERE cu.CustomerID NOT IN (
    SELECT DISTINCT CustomerID FROM "Order" WHERE CustomerID IS NOT NULL
)
GROUP BY cu.CustomerID, cu.CustomerName, cu.Email, cu.Phone;

-- 1.4 Each customer's default delivery address
SELECT cu.CustomerName, a.Street, a.City, a.State, a.Pincode
FROM Customer      cu
JOIN CustomerAddress ca ON cu.CustomerID = ca.CustomerID
JOIN Address        a  ON ca.AddressID  = a.AddressID
WHERE ca.IsDefault = TRUE
ORDER BY cu.CustomerName;

-- =====
-- SCENARIO 2: Order Management

```

-- "Track every order from placement to delivery"

--

-- 2.1 Complete order details with customer, coupon, payment, shipment

```
SELECT cu.CustomerName, o.OrderID, o.OrderDate, o.TotalAmount,
       o.Status AS OrderStatus,
       COALESCE(co.Code, 'No Coupon') AS CouponUsed,
       pay.GateWayName, pay.Status AS PaymentStatus,
       s.CourierPartner, s.Tracking, s.Status AS ShipmentStatus
FROM "Order"      o
JOIN Customer     cu ON o.CustomerID = cu.CustomerID
LEFT JOIN Coupon  co ON o.CouponID   = co.CouponID
LEFT JOIN Payment pay ON o.OrderID    = pay.OrderID
LEFT JOIN Shipment s  ON o.OrderID    = s.OrderID
ORDER BY o.OrderDate;
```

-- 2.2 Only delivered orders with courier and delivery date

```
SELECT cu.CustomerName, o.OrderID, o.TotalAmount,
       s.CourierPartner, s.DispatchDate, s.ETA AS DeliveredOn
FROM "Order" o
JOIN Customer cu ON o.CustomerID = cu.CustomerID
JOIN Shipment s  ON o.OrderID    = s.OrderID
WHERE o.Status = 'Delivered'
ORDER BY s.ETA;
```

-- 2.3 Orders with failed or pending payments (need follow-up)

```
SELECT cu.CustomerName, cu.Email, cu.Phone,
       o.OrderID, pay.Amount, pay.GateWayName,
       pay.TxnRef, pay.Status AS PaymentStatus
FROM Payment pay
JOIN "Order" o ON pay.OrderID = o.OrderID
JOIN Customer cu ON o.CustomerID = cu.CustomerID
WHERE pay.Status IN ('Failed', 'Pending')
ORDER BY o.OrderID;
```

-- 2.4 Order count and total revenue grouped by status

```
SELECT Status,
       COUNT(*) AS TotalOrders,
       SUM(TotalAmount) AS TotalRevenue
FROM "Order"
GROUP BY Status
ORDER BY TotalOrders DESC;
```

-- 2.5 Actual discount received per order using coupon

```
SELECT cu.CustomerName, o.OrderID, o.TotalAmount,
       co.Code, co.DiscountType, co.DiscountValue,
       CASE
         WHEN co.DiscountType = 'Flat'
           THEN co.DiscountValue
         WHEN co.DiscountType = 'Percentage'
```

```

            THEN ROUND((o.TotalAmount * co.DiscountValue / 100), 2)
        END AS ActualDiscount
FROM "Order" o
JOIN Customer cu ON o.CustomerID = cu.CustomerID
JOIN Coupon co ON o.CouponID = co.CouponID
ORDER BY ActualDiscount DESC;

```

---

```

-- SCENARIO 3: Inventory & Warehouse Management
-- "Know what stock is where and what needs reordering"

```

---

```

-- 3.1 Total stock in each warehouse vs its capacity
SELECT w.WarehouseName, w.City, w.State,
       SUM(i.Quantity) AS TotalStock,
       w.Capacity,
       (w.Capacity - SUM(i.Quantity)) AS RemainingCapacity
FROM Warehouse w
JOIN Inventory i ON w.WarehouseID = i.WarehouseID
GROUP BY w.WarehouseID, w.WarehouseName, w.City, w.State, w.Capacity
ORDER BY TotalStock DESC;

```

```

-- 3.2 Products whose stock has fallen below reorder level
SELECT p.ProductName, pv.Color, pv.Size,
       w.WarehouseName, w.City,
       i.Quantity AS CurrentStock,
       i.OrderLevel AS ReorderLevel
FROM Inventory i
JOIN ProductVariant pv ON i.VariantID = pv.VariantID
JOIN Product p ON pv.ProductID = p.ProductID
JOIN Warehouse w ON i.WarehouseID = w.WarehouseID
WHERE i.Quantity <= i.OrderLevel
ORDER BY i.Quantity ASC;

```

```

-- 3.3 Full inventory list with product, warehouse and bin location
SELECT w.WarehouseName, w.City, p.ProductName, p.Brand,
       pv.Color, pv.Size, i.Quantity, i.BinLocation
FROM Inventory i
JOIN Warehouse w ON i.WarehouseID = w.WarehouseID
JOIN ProductVariant pv ON i.VariantID = pv.VariantID
JOIN Product p ON pv.ProductID = p.ProductID
ORDER BY w.WarehouseName, p.ProductName;

```

---

```

-- SCENARIO 4: Vendor & Purchase Order Management
-- "Track what was ordered from which vendor and its status"

```

---

-- 4.1 All purchase orders with vendor name and product details

```
SELECT v.CompanyName, po.PO_ID, po.OrderDate,
       po.Status, po.ExpDelivDate,
       p.ProductName, pv.Color, pv.Size, poi.Quantity
FROM PurchaseOrder po
JOIN Vendor v ON po.VendorID = v.VendorID
JOIN PurchaseOrderItem poi ON po.PO_ID = poi.PO_ID
JOIN VendorProduct vp ON poi.VP_ID = vp.VP_ID
JOIN ProductVariant pv ON vp.VariantID = pv.VariantID
JOIN Product p ON pv.ProductID = p.ProductID
ORDER BY po.OrderDate;
```

-- 4.2 Purchase orders still not delivered (pending/confirmed/shipped)

```
SELECT v.CompanyName, po.PO_ID, p.ProductName,
       poi.Quantity, po.ExpDelivDate, po.Status
FROM PurchaseOrder po
JOIN Vendor v ON po.VendorID = v.VendorID
JOIN PurchaseOrderItem poi ON po.PO_ID = poi.PO_ID
JOIN VendorProduct vp ON poi.VP_ID = vp.VP_ID
JOIN ProductVariant pv ON vp.VariantID = pv.VariantID
JOIN Product p ON pv.ProductID = p.ProductID
WHERE po.Status IN ('Pending', 'Confirmed', 'Shipped')
ORDER BY po.ExpDelivDate;
```

-- 4.3 Top vendors by rating (4.0 and above)

```
SELECT CompanyName, ContactPerson, Rating, Email, Phone
FROM Vendor
WHERE Rating >= 4.0
ORDER BY Rating DESC;
```

-- 4.4 How many different product variants each vendor supplies

```
SELECT v.CompanyName, v.Rating,
       COUNT(DISTINCT vp.VariantID) AS TotalVariantsSupplied
FROM Vendor v
JOIN VendorProduct vp ON v.VendorID = vp.VendorID
GROUP BY v.VendorID, v.CompanyName, v.Rating
ORDER BY TotalVariantsSupplied DESC;
```

---

---

-- SCENARIO 5: Sales & Revenue Analysis  
-- "Understand where money is coming from"

---

---

-- 5.1 Total revenue generated per product category

```
SELECT c.CategoryName,
       COUNT(DISTINCT oi.OrderItemID) AS TotalItemsSold,
       SUM(oi.Quantity * oi.UniPrice) AS TotalRevenue
FROM OrderItem oi
JOIN VendorProduct vp ON oi.VP_ID = vp.VP_ID
```

```

JOIN ProductVariant pv ON vp.VariantID = pv.VariantID
JOIN Product      p  ON pv.ProductID = p.ProductID
JOIN Category     c  ON p.CategoryID = c.CategoryID
GROUP BY c.CategoryName
ORDER BY TotalRevenue DESC;

```

-- 5.2 Top spending customers (excluding cancelled orders)

```

SELECT cu.CustomerName, cu.Email,
       COUNT(o.OrderID)          AS TotalOrders,
       SUM(o.TotalAmount)        AS TotalSpent,
       ROUND(AVG(o.TotalAmount), 2) AS AvgOrderValue
FROM Customer cu
JOIN "Order" o ON cu.CustomerID = o.CustomerID
WHERE o.Status != 'Cancelled'
GROUP BY cu.CustomerID, cu.CustomerName, cu.Email
ORDER BY TotalSpent DESC;

```

-- 5.3 Payment gateway wise total collection from successful payments

```

SELECT GateWayName,
       COUNT(*)      AS TotalTransactions,
       SUM(Amount)    AS TotalCollected
FROM Payment
WHERE Status = 'Success'
GROUP BY GateWayName
ORDER BY TotalCollected DESC;

```

-- 5.4 Best selling products by total units sold

```

SELECT p.ProductName, p.Brand,
       SUM(oi.Quantity)          AS TotalUnitsSold,
       SUM(oi.Quantity * oi.UniPrice) AS TotalRevenue
FROM OrderItem oi
JOIN VendorProduct vp ON oi.VP_ID = vp.VP_ID
JOIN ProductVariant pv ON vp.VariantID = pv.VariantID
JOIN Product p ON pv.ProductID = p.ProductID
GROUP BY p.ProductID, p.ProductName, p.Brand
ORDER BY TotalUnitsSold DESC;

```

-- 5.5 Coupon usage count and how much discount was given

```

SELECT co.Code, co.DiscountType, co.DiscountValue,
       COUNT(o.OrderID) AS TimesUsed,
       co.UsageLimit,
       (co.UsageLimit - COUNT(o.OrderID)) AS RemainingUses
FROM Coupon co
LEFT JOIN "Order" o ON co.CouponID = o.CouponID
GROUP BY co.CouponID, co.Code, co.DiscountType,
         co.DiscountValue, co.UsageLimit
ORDER BY TimesUsed DESC;

```

--

---

```
-- SCENARIO 6: Return & Refund Management
-- "Handle what came back and how much was refunded"
-- =====
```

```
-- 6.1 All return requests with customer name and product detail
```

```
SELECT rr.ReturnID, cu.CustomerName, p.ProductName,
       rr.Reason, rr.Status, rr.RefundAmount, rr.RegDate
FROM ReturnRefund rr
JOIN OrderItem oi ON rr.OrderItemID = oi.OrderItemID
JOIN "Order" o ON oi.OrderID = o.OrderID
JOIN Customer cu ON o.CustomerID = cu.CustomerID
JOIN VendorProduct vp ON oi.VP_ID = vp.VP_ID
JOIN ProductVariant pv ON vp.VariantID = pv.VariantID
JOIN Product p ON pv.ProductID = p.ProductID
ORDER BY rr.RegDate;
```

```
-- 6.2 Total amount already refunded to customers
```

```
SELECT SUM(RefundAmount) AS TotalRefundedAmount
FROM ReturnRefund
WHERE Status = 'Refunded';
```

```
-- 6.3 Return count and total refund grouped by reason
```

```
SELECT Reason,
       COUNT(*) AS TotalReturns,
       SUM(RefundAmount) AS TotalRefundGiven
FROM ReturnRefund
GROUP BY Reason
ORDER BY TotalReturns DESC;
```

```
-- 6.4 Returns that are approved but refund not yet processed
```

```
SELECT rr.ReturnID, cu.CustomerName, cu.Email,
       p.ProductName, rr.RefundAmount, rr.RegDate
FROM ReturnRefund rr
JOIN OrderItem oi ON rr.OrderItemID = oi.OrderItemID
JOIN "Order" o ON oi.OrderID = o.OrderID
JOIN Customer cu ON o.CustomerID = cu.CustomerID
JOIN VendorProduct vp ON oi.VP_ID = vp.VP_ID
JOIN ProductVariant pv ON vp.VariantID = pv.VariantID
JOIN Product p ON pv.ProductID = p.ProductID
WHERE rr.Status = 'Approved'
ORDER BY rr.RegDate;
```

```
-- =====
-- SCENARIO 7: Product & Review Analysis
-- "Which products are loved and which are not"
-- =====
```

```
-- 7.1 All reviews with customer name and product name
```

```
SELECT cu.CustomerName, p.ProductName, pv.Color,
```

```

        r.Rating, r.Comment, r.ReviewDate
FROM Review      r
JOIN Customer    cu ON r.CustomerID = cu.CustomerID
JOIN VendorProduct vp ON r.VP_ID      = vp.VP_ID
JOIN ProductVariant pv ON vp.VariantID = pv.VariantID
JOIN Product     p  ON pv.ProductID = p.ProductID
ORDER BY r.Rating DESC;

```

-- 7.2 Average rating per product (best rated first)

```

SELECT p.ProductName, p.Brand,
       ROUND(AVG(r.Rating), 2) AS AvgRating,
       COUNT(r.ReviewID)      AS TotalReviews
FROM Review      r
JOIN VendorProduct vp ON r.VP_ID      = vp.VP_ID
JOIN ProductVariant pv ON vp.VariantID = pv.VariantID
JOIN Product     p  ON pv.ProductID = p.ProductID
GROUP BY p.ProductID, p.ProductName, p.Brand
ORDER BY AvgRating DESC;

```

-- 7.3 Products with low ratings (below 3) - need attention

```

SELECT p.ProductName, cu.CustomerName,
       r.Rating, r.Comment, r.ReviewDate
FROM Review      r
JOIN Customer    cu ON r.CustomerID = cu.CustomerID
JOIN VendorProduct vp ON r.VP_ID      = vp.VP_ID
JOIN ProductVariant pv ON vp.VariantID = pv.VariantID
JOIN Product     p  ON pv.ProductID = p.ProductID
WHERE r.Rating < 3
ORDER BY r.Rating ASC;

```